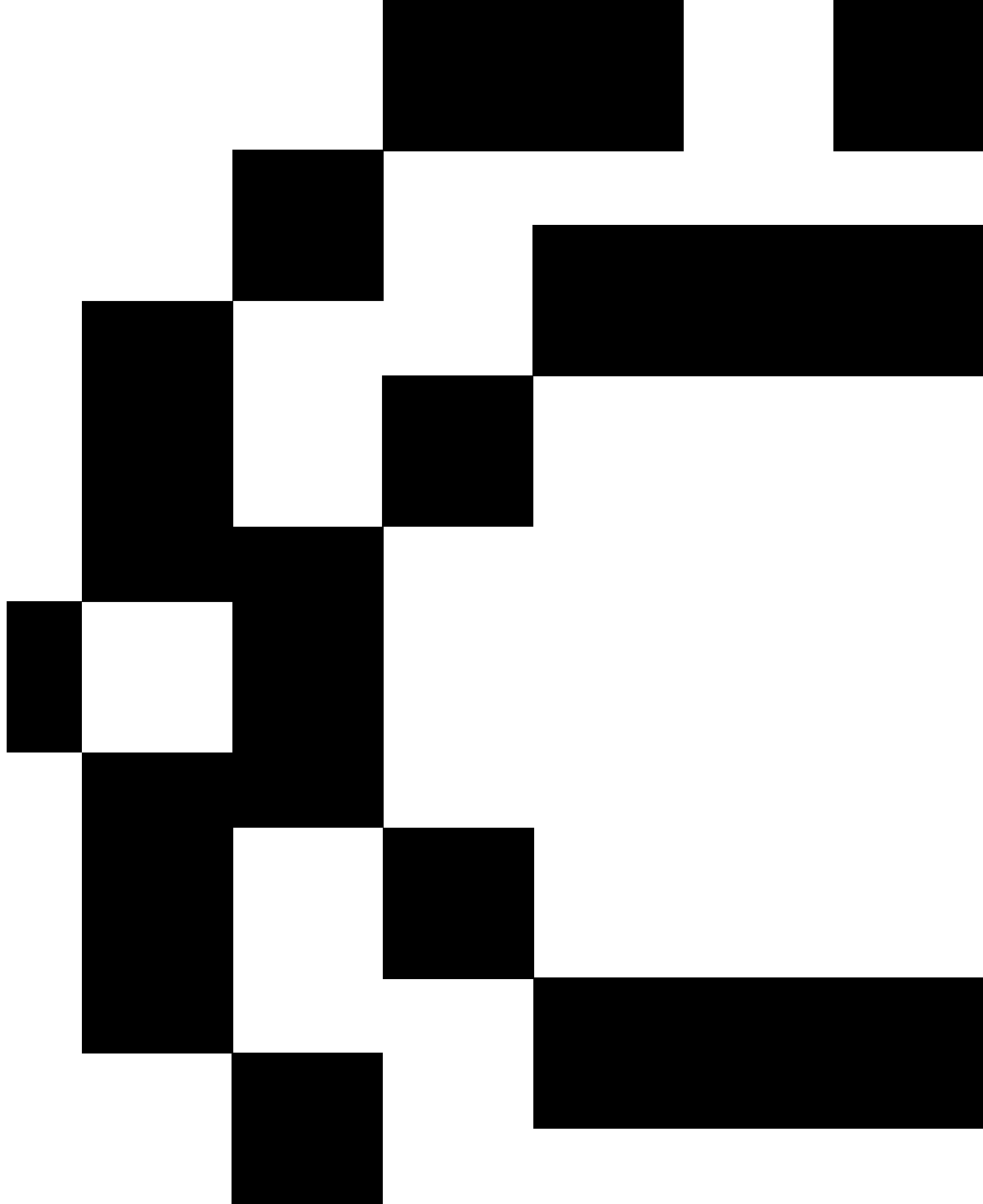
NOVA IMS INFORMATION MANAGEMENT SCHOOL

Computation I / Introduction to Programming

Practicals Week 08 - While Loops

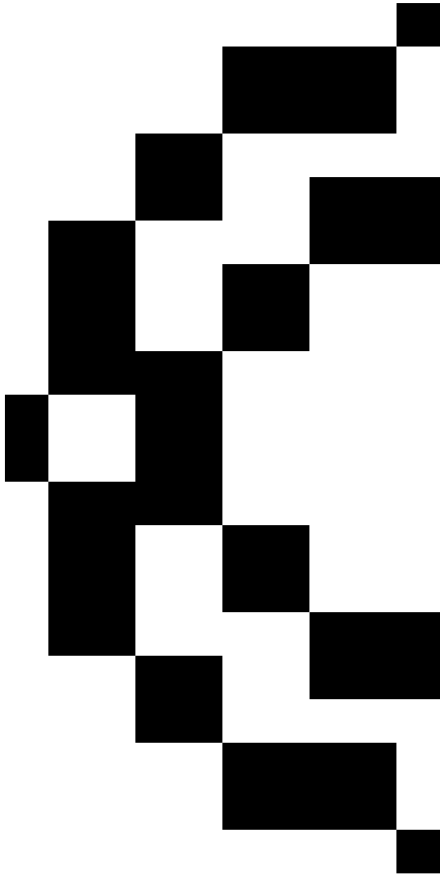


Exercise 1

Using a `while` loop, print all the odd numbers smaller than 10.

Example

```
Input:  (no input)
Output: 1
        3
        5
        7
        9
```



Exercise 1 — Solution

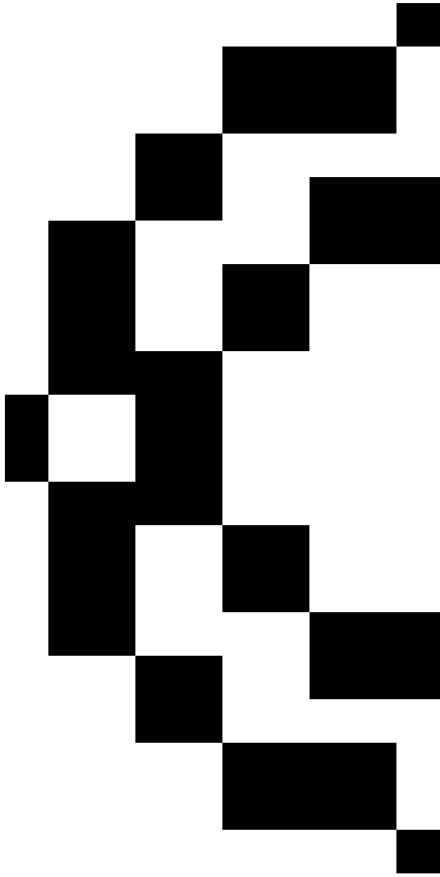
```
x = 1
while x < 10:
    print(x)
    x += 2
```

Exercise 2

Print the multiplication table of 2 up to 10. Your program should print `"2x1=2"`, `"2x2=4"`, and so on, all the way up to `"2x10=20"`.

Example

```
Input:  (no input)
Output: 2x1=2
        2x2=4
        ...
        2x10=20
```



Exercise 2 — Solution

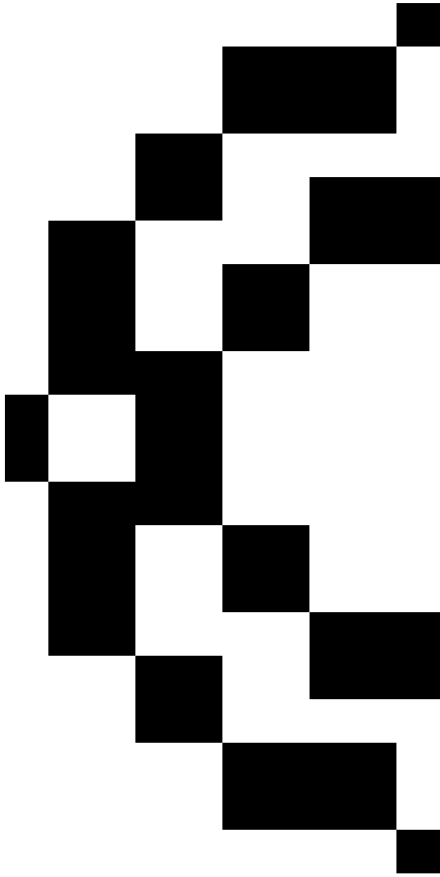
```
x = 1
while x <= 10:
    print(f"2x{x}={2*x}")
    x += 1
```

Exercise 3

Define an integer of your choice between 1 and 10. Ask the user to guess the number, and keep asking for new guesses with a `while` loop until they guess correctly.

Example

- `Input:  number = 8, guesses = 3, 5, 8`
- `Output: Congrats!`



Exercise 3 — Solution

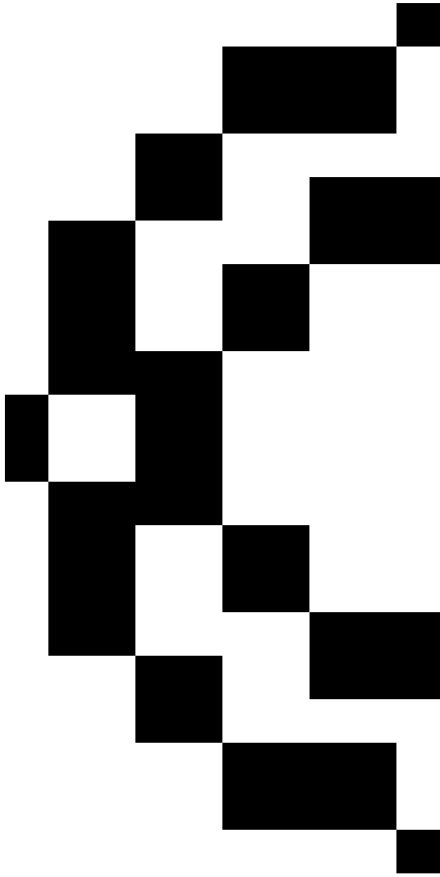
```
number = 8
guess = int(input("Your guess: "))
while guess != number:
    guess = int(input("Nope. Try again: "))
print("Congrats!")
```

Exercise 4

Ask the user how many integers they would like to insert. Then receive that many integers, one at a time, and for each one print whether it is even or odd.

Example

```
Input:  n = 3, numbers = 4, 7, 10
Output: Even
        Odd
        Even
```



Exercise 4 — Solution

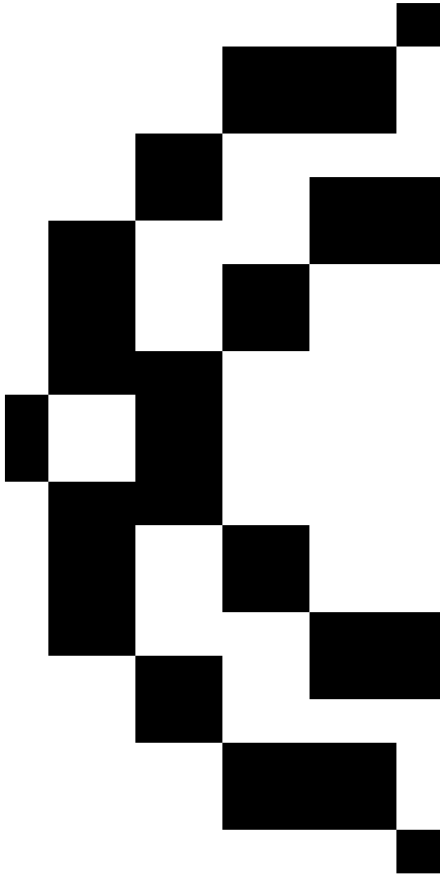
```
n = int(input("How many numbers do you want to insert? "))
i = 1
while i <= n:
    x = int(input(f"Insert number {i}"))
    if x % 2 == 0:
        print("Even")
    else:
        print("Odd")
    i += 1
```

Exercise 5

Write a program that lets the user keep entering numbers. Add each number to a running total and display the total after every entry. Stop asking for numbers once the total exceeds 100.

Example

```
Input:  numbers = 40, 35, 30
Output: 40.0
        75.0
        105.0
        That's all folks!
```



Exercise 5 — Solution

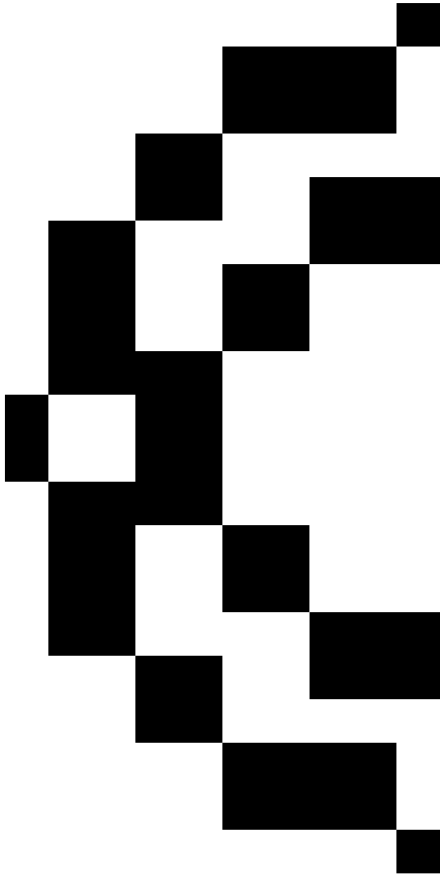
```
my_sum = 0
while my_sum <= 100:
    number = float(input("Insert a number"))
    my_sum += number
    print(my_sum)
print("That's all folks!")
```

Exercise 6

Ask the user for two integers. Calculate the result of the first number raised to the power of the second, without using the `**` operator or the `math` module — using loops only.

Example

- `Input:  base = 2, exp = 5`
- `Output: 32`



Exercise 6 — Solution

```
base = int(input("Insert a number for the base"))
exp = int(input("Insert a number for the exponent"))

i = 1
result = 1
while i <= exp:
    result *= base
    i += 1

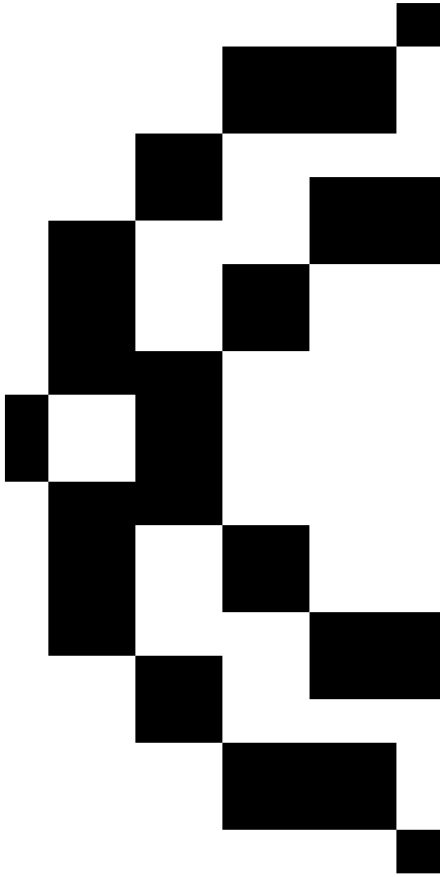
print(result)
```

Exercise 7

Define a list called `my_list` with the value `[1, 2, 3]`. Allow the user to keep adding integers to this list, printing the updated list and its average after each addition, until the average of the list becomes higher than 5.

Example

- `Input:  my_list = [1, 2, 3], additions = 20`
- `Output: New list is [1, 2, 3, 20], average is 6.5`



Exercise 7 — Solution

```
my_list = [1, 2, 3]
total = 0
for number in my_list:
    total += number

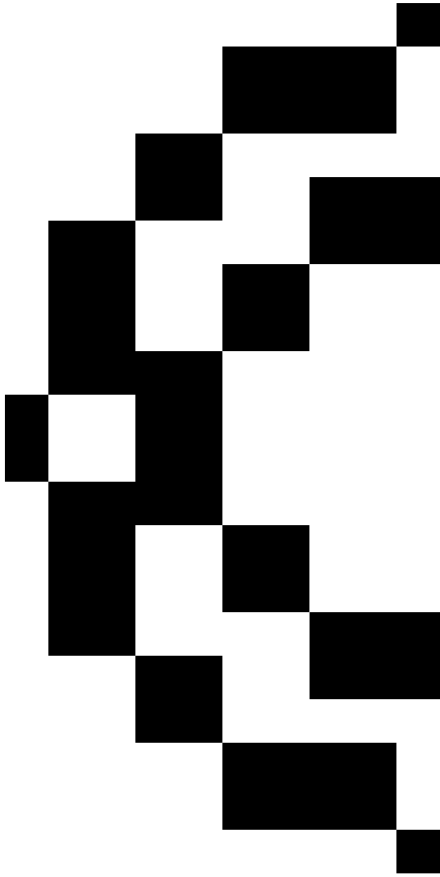
while total / len(my_list) <= 5:
    new_number = int(input("Your number: "))
    my_list.append(new_number)
    total += new_number
    print(f"New list is {my_list}, average is {total / len(my_list)}")
```

Exercise 8

Extend the number-guessing game from Exercise 3: keep track of how many guesses the user makes. If they need 2 guesses or fewer, congratulate them genuinely. If they need between 3 and 5 guesses (inclusive), tell them it was not very impressive. If they need more than 5, tell them they have zero luck today.

Example

- `Input:  number = 8, guesses = 3, 5, 8`
- `Output: Not very impressive...`



Exercise 8 — Solution

```
number = 8
guess = int(input("Your guess: "))
guess_counter = 1
while guess != number:
    guess = int(input("Nope. Try again: "))
    guess_counter += 1

if guess_counter <= 2:
    print("Awesome, well done!")
elif 2 < guess_counter < 6:
    print("Not very impressive...")
else:
    print("You don't have luck today :(")
```

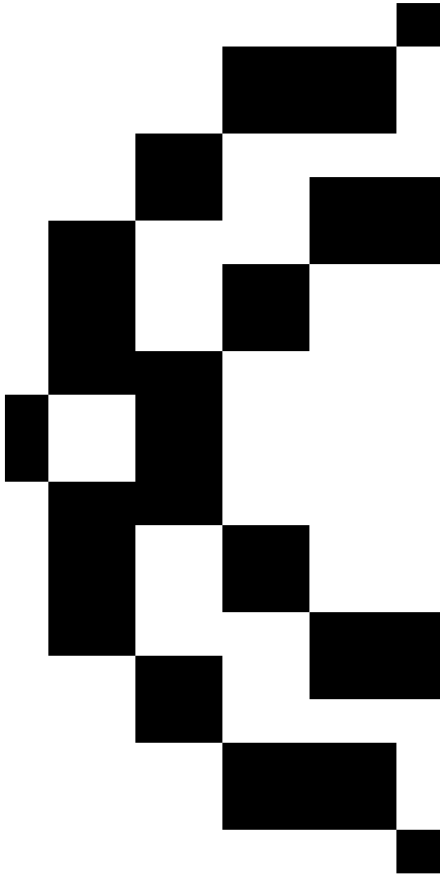
Exercise 9

Write a program that reads a string from the user. Print `"Capital letter detected." if the string contains at least one capital letter, and "No capital letters."` otherwise.

Examples

- Input: `user_string = "Hello world"`
- Output: `Capital letter detected.`

- Input: `user_string = "hello world"`
- Output: `No capital letters.`



Exercise 9 — Solution

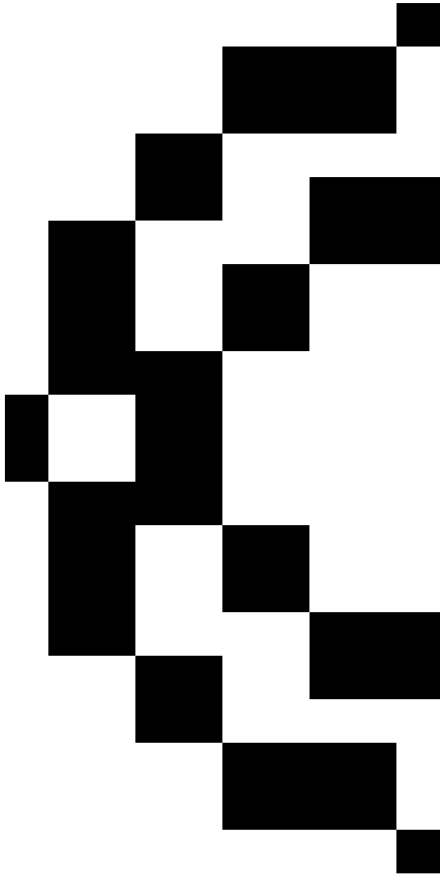
```
user_string = input("Say something: ")
if user_string != user_string.lower():
    print("Capital letter detected.")
else:
    print("No capital letters.")
```

Exercise 10

Write a program that asks the user for a password. Keep asking until the user enters a password that contains at least two upper-case letters.

Example

- Input: `attempts = "password", "Password", "PassWord"`
- Output: `(loop ends after "PassWord" is entered)`



Exercise 10 — Solution

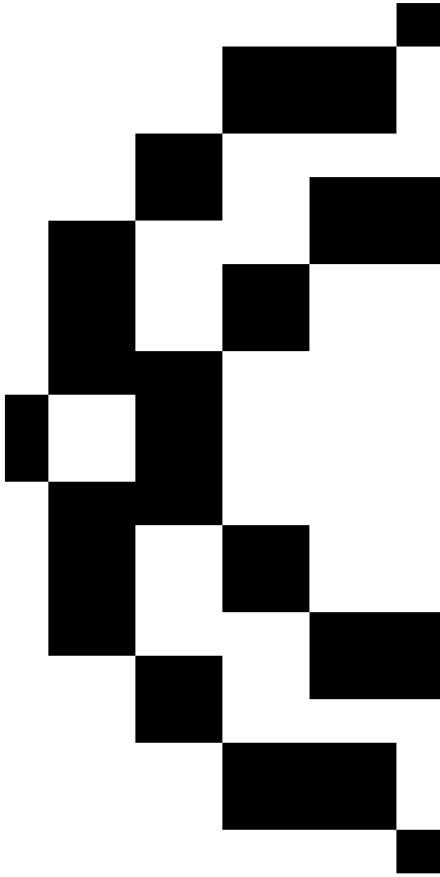
```
caps = 0
while caps < 2:
    password = input("Gimme your password: ")
    caps = 0
    for letter in password:
        if letter.isupper():
            caps += 1
```

Exercise 11

You are given a secret, encoded message: `'blephatxyzrs gotwo_na hatxyzgo'`. It was encoded as follows: every `"e"` was replaced with `"xyz"`; the text `"blep"` was added to the beginning and `"go"` was added to the end; and every `"nn"` was replaced with `"two_n"`. Reverse these steps, in the correct order, to recover and print the original message.

Example

- `Input:  secret = 'blephatxyzrs gotwo_na hatxyzgo'`
- `Output: haters gonna hate`



Exercise 11 — Solution

```
secret = 'blephatxyzrs gotwo_na hatxyzgo'
secret = secret.replace("xyz", "e")
secret = secret[4:-2]
secret = secret.replace("two_n", "nn")
print(secret)
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

Thank you

Acreditações e Certificações da NOVA IMS

Cofinanciado por